

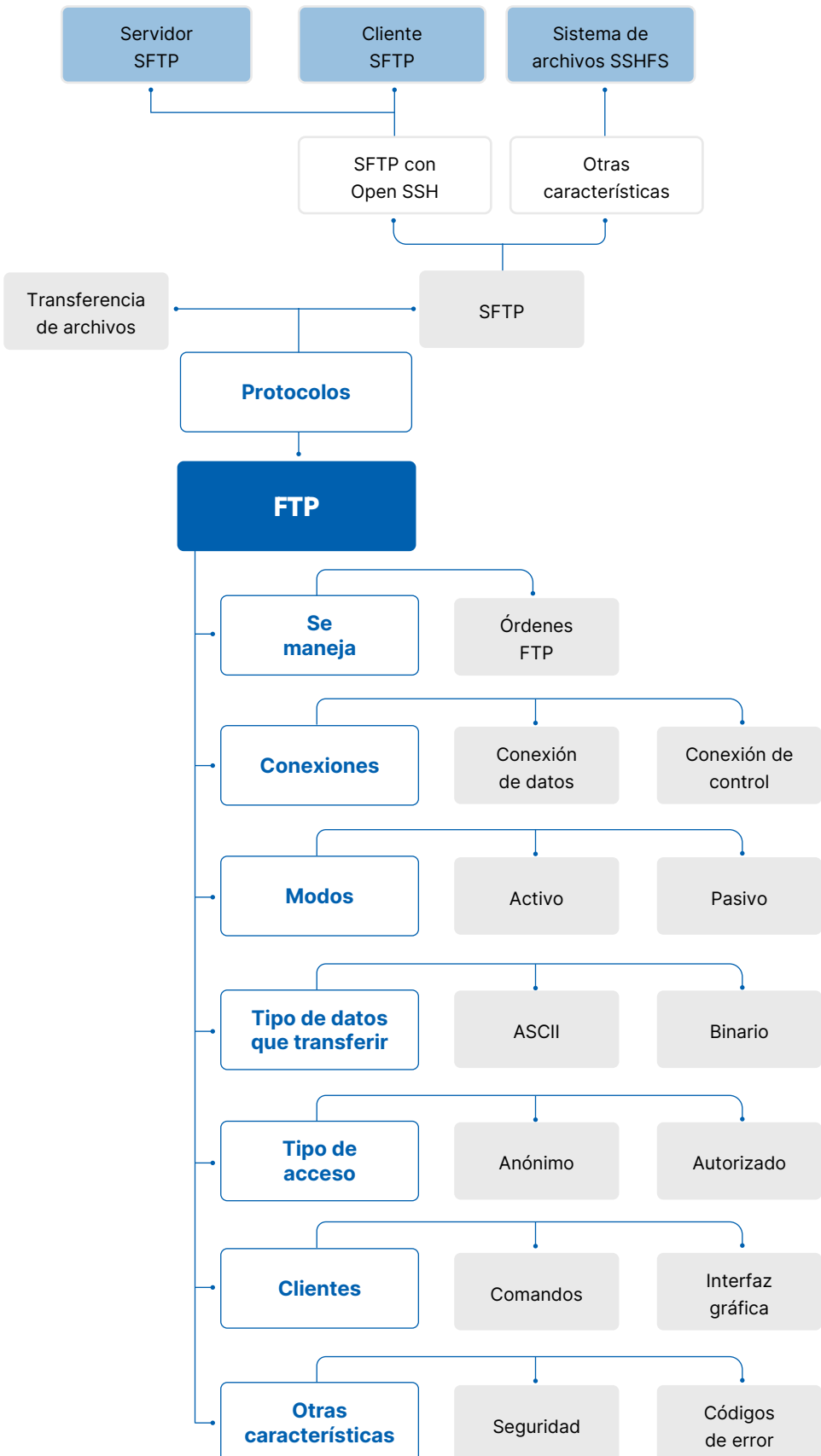
UF4

Transferencia de archivos

Despliegue de aplicaciones web

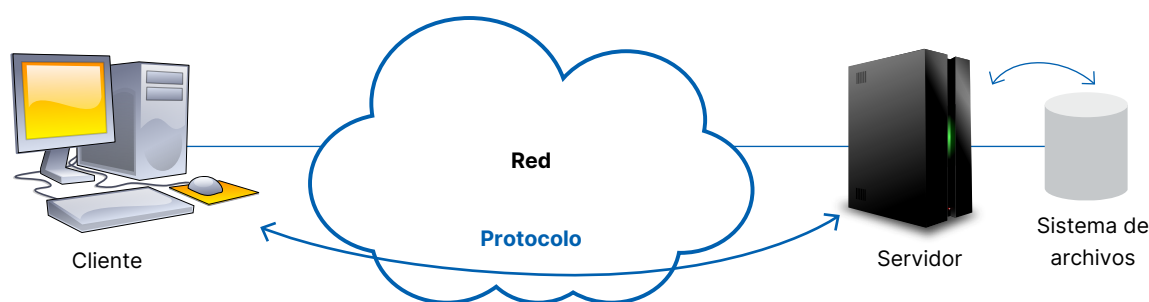
ÍNDICE

Mapa conceptual	03
1. Introducción a la transferencia de archivos	04
1.1. Introducción a FTP.....	05
1.2. Introducción a SFTP.....	05
2. Protocolo FTP.....	09
2.1. Funcionamiento	09
2.2. Modos de conexión	10
A. Modo activo.....	10
B. Modo pasivo	11
2.3. Transferencia de información	11
2.4. Códigos de respuesta	12
2.5. Tipo de acceso.....	13
2.6. Clientes	13
2.7. Servidores.....	15
2.7. Seguridad	15
3. Protocolo SFTP	21
3.1. Funcionamiento de SFTP	23
3.2. SFTP con OpenSSH.....	24
3.3. Sistema de archivos SSHFS	28



01 Introducción a la transferencia de archivos

Uno de los principales usos de las redes de ordenadores es el **almacenamiento remoto** de archivos, que permite alojar información en otros equipos a través de la red. Aunque existen otros modelos, normalmente se usa el cliente-servidor para el diseño de este tipo de almacenamiento. Sus elementos básicos son: el programa cliente, el servidor, el protocolo de **transferencia de archivos** y el sistema de archivos del servidor.



Esquema 1.

Existen innumerables **protocolos de transferencia de archivos** que permiten el intercambio de archivos entre equipos a través de la red. Estos son algunos de los más importantes:

FTP	File Transfer Protocol	(RFC 959)
FTPS	File Transfer Protocol over TLS/SSL	(RFC 4217)
SFTP	SSH File Transfer Protocol	(RFC en progreso)
TFTP	Trivial File Transfer Protocol	(RFC 1350)
NFS	Network File System	(RFC 7862)
WebDAV	Web Distributed Authoring and Versioning	(RFC 4918)
Samba	Implementación libre de SMB/CIFS	(Samba)
rsync	Remote synchronization	(Andrew Tridgell y Paul Mackerras)
SMB/CIFS	Server Message Block/Common Internet File System	(Microsoft)

Tabla 1.

Estos protocolos son implementados por los **programas de transferencia de archivos**, que pueden ser aplicaciones específicas para este propósito o que hayan sido diseñadas para otro propósito, pero que también permiten el intercambio de archivos entre distintas máquinas. Por ejemplo:

- » FileZilla, que implementa tanto un cliente como un servidor, es capaz de usar los protocolos FTP, FTPS y SFTP.
- » El gestor de archivos Nautilus, que permite trabajar con la mayoría de los protocolos descritos en este apartado.

Por otro lado, la posibilidad de montar o conectar directorios remotos como si formaran parte del sistema de archivos local es muy interesante, porque el usuario, gracias a los gestores de archivos, puede abrir dichos directorios y editar sus archivos con los programas instalados en su propio equipo. De esta manera, la transferencia de archivos es totalmente transparente para él.

1.1. Introducción a FTP

El protocolo FTP nació con el propósito de poder transferir datos entre ordenadores de forma eficiente y fiable. De este modo, se fomenta la utilización del almacenamiento de información de forma remota y se promueve el concepto de compartir información.

Desde la primera versión, desarrollada para usarse en servidores del MIT (Massachusetts Institute of Technology), el protocolo FTP ha ido añadiendo características para poder cumplir cada vez mejor ese objetivo. Versión tras versión, se ha optimizado la documentación, en la que se han corregido errores y se ha mejorado la explicación de algunas características. Además, se han añadido algunas órdenes opcionales nuevas.

El funcionamiento, a rasgos generales, se puede describir de la siguiente forma: el cliente o usuario conecta con el servidor a través de una interfaz, que puede ser una sesión de terminal o un programa con entorno gráfico. Una vez establecida la conexión, servidor y cliente son capaces de transferir archivos en ambas direcciones o solamente en una, dependiendo de los permisos que tenga el usuario.

1.2. Introducción a SFTP

El protocolo de transferencia de archivos **SFTP** es conocido como *SSH File Transfer Protocol* y, a pesar de su nombre, no es un derivado ni de FTP ni de SSH, aunque casi siempre lo encontraremos junto a este último protocolo, ya que **OpenSSH** lo incluye en la instalación de su servidor SSH. Gracias a esto, es ampliamente utilizado, a pesar de que aún se trata de un borrador IETF y no dispone de un RFC.

SFTP permite implementar tanto un sistema de archivos remoto como un servicio de transferencia remota de archivos. Hay que tener en cuenta que este protocolo no se encarga ni de la autenticación ni de la seguridad de los datos transferidos, sino que asume que trabaja sobre un canal seguro, como SSH, TLS o VPN.

1

CASO PRÁCTICO



INSTALACIÓN DE UN SERVIDOR FTP

En este caso práctico vas a instalar un servidor FTP. Hay varios servidores de código abierto y en este caso se va a utilizar el servidor *vsftpd*. Después de instalarlo, has de configurar el acceso anónimo y comprobar que conecta desde un terminal o un explorador de archivos.

SOLUCIÓN

1. Accede al servidor con un usuario administrador. Actualiza la lista de paquetes disponibles en los repositorios e instala *vsftpd*.

```
$ ssh admin01@192.168.1.100
admin01@192.168.1.100's password:
Welcome to Ubuntu 22.04 LTS (GNU/Linux 5.15.0-37-generic x86_64)
[...]
admin01@srv:~$ sudo apt-get update
Obj:1 http://es.archive.ubuntu.com/ubuntu jammy InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu jammy-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu jammy-backports InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
admin01@srv:~$ sudo apt-get install vsftpd
[...]
admin01@srv:~$
```

2. Comprueba si el cortafuegos de Ubuntu está activo y, en caso afirmativo, configúralo adecuadamente.

```
admin01@srv:~$ sudo ufw status
```

```
Status: inactive
```

```
admin01@srv:~$
```

Si el resultado es *Status: inactive*, no has de hacer nada.

Si el resultado es *Status: active*, has de permitir el tráfico por los puertos 20 y 21 con los comandos:

```
sudo ufw allow 20/tcp
```

```
sudo ufw allow 21/tcp
```

3. Crea el directorio llamado *ftp* dentro del directorio */var*, que será el directorio al que accederá el usuario.

```
admin01@srv:~$ sudo mkdir /var/ftp
```

4. Crea el archivo *pruebas.txt* dentro del directorio *ftp* que acabas de crear con el contenido que quieras.

```
admin01@srv:~$ cd /var/ftp
```

```
admin01@srv:~$ sudo nano pruebas.txt
```

5. Haz copia del archivo original de configuración del servidor ftp que acabas de instalar.

```
admin01@srv:~$ cd /etc
```

```
admin01@srv:~$ sudo cp vsftpd.conf vsftpd-bk.conf
```

6. Edita el archivo *vsftpd.conf*.

```
admin01@srv:~$ sudo nano vsftpd.conf
```

7. Localiza la línea *anonymous_enable=NO* y coméntala, después añade debajo las siguientes líneas, de forma que quede así:

```
# anonymous_enable=NO
anonymous_enable=YES
anon_root=/var/ftp
anon_world_readable_only=YES
anon_upload_enable=NO
anon_mkdir_write_enable=NO
```

Fíjate que el directorio al que tendrá acceso el usuario anónimo es */var/ftp* y que solamente tiene permisos de lectura, ya que no se permite subir archivos ni crear directorios.

8. Reinicia el servicio y comprueba que se haya levantado correctamente.

```
admin01@srv:~$ sudo systemctl restart vsftpd.service
```

```
admin01@srv:~$ sudo systemctl status vsftpd.service
```

- `vsftpd.service` - vsftpd FTP server

Loaded: loaded (/lib/systemd/system/vsftpd.service; enabled; vendor preset: enabled)

Active: active (running) since Tue 2022-07-19 18:34:12 UTC; 6s ago

Process: 2015 ExecStartPre=/bin/mkdir -p /var/run/vsftpd/empty (code=exited, status=0/SUCCESS)

Main PID: 2016 (vsftpd)

Tasks: 1 (limit: 4579)

Memory: 852.0K

CPU: 3ms

CGroup: /system.slice/vsftpd.service

└─ 2016 /usr/sbin/vsftpd /etc/vsftpd.conf

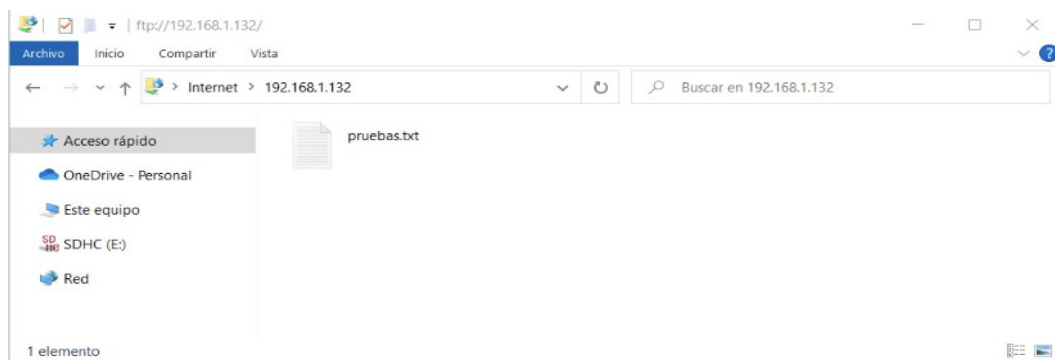
```
jul 19 18:34:12 server systemd[1]: Starting vsftpd FTP server...
```

```
jul 19 18:34:12 server systemd[1]: Started vsftpd FTP server.
```

```
admin01@server:~$
```

9. Comprueba que puedes acceder y que ves el archivo *pruebas.txt* que has creado anteriormente. Para establecer la conexión, puedes utilizar una ventana del explorador de archivos e indicar en la ruta: *ftp://IP_servidor*.

Aquí tienes un ejemplo realizado en el explorador de archivos de Windows 10:

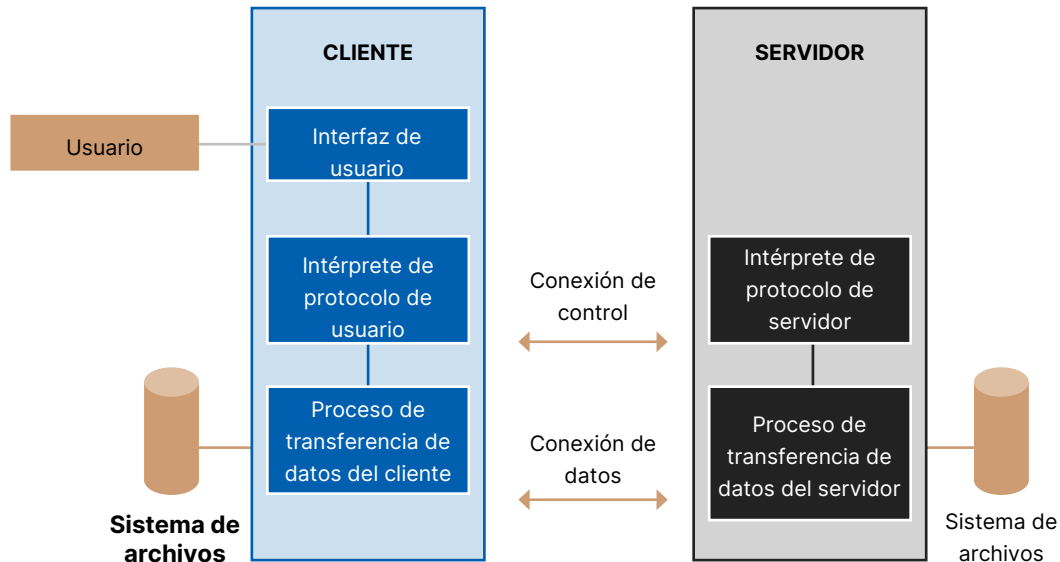


02 Protocolo FTP

Como se ha visto en la introducción de esta unidad, para que la transferencia de archivos se lleve a cabo, una máquina debe actuar como servidor y otra como cliente. El servidor debe estar configurado a la escucha y listo para recibir las conexiones de los clientes, permitir o no dichas conexiones y, en el caso de haberse establecido la conexión con éxito, ser capaz de, por un lado, localizar los archivos solicitados por el cliente y enviarlos, y, por otro lado, almacenar correctamente los archivos recibidos por el cliente. Este último es el que solicita al servidor una conexión y realiza las peticiones sobre los archivos que quiere enviar o recibir.

2.1. Funcionamiento

Servidor y cliente deben estar comunicados en todo momento para que el servidor sea capaz de procesar correctamente las órdenes que el cliente le envía. En el siguiente esquema, se puede observar una representación gráfica de esta explicación:



Esquema 2.

El protocolo FTP necesita dos conexiones para funcionar, la conexión de datos, por la que se transmiten los archivos, y la conexión de control, por la que se transmiten las órdenes. A continuación, se describen sus funciones:

Las órdenes FTP

Son las que contienen información sobre cómo tiene que comportarse la comunicación. Esta información puede ser sobre la conexión de datos (por ejemplo, para indicar el puerto de conexión o el modo de transferencia) o también puede ser sobre la operación que realizar en el sistema de archivos (por ejemplo, para indicar si es una operación de escritura, lectura o borrado en disco).

La conexión de control

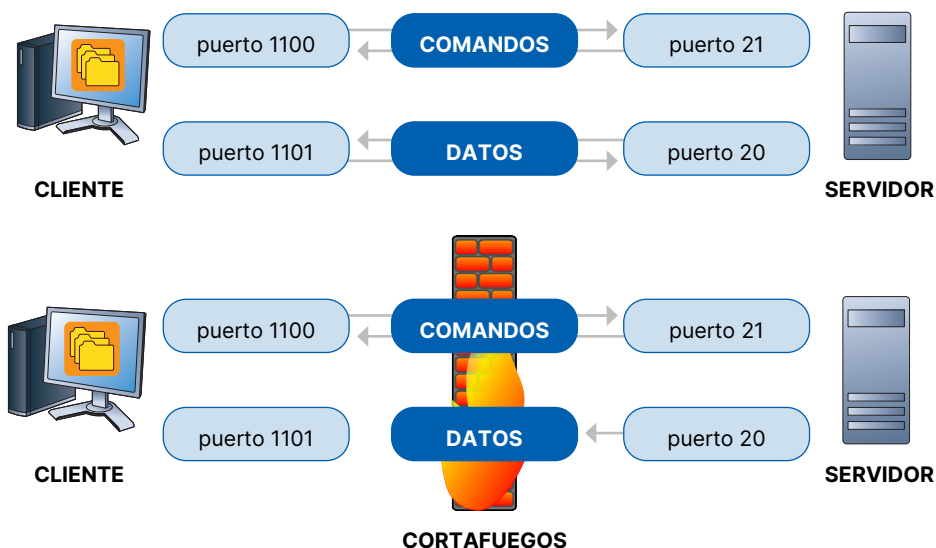
Es por la que viajan las órdenes que envía el intérprete de protocolo de usuario y las respuestas que envía el intérprete de protocolo de servidor a esas peticiones. Por defecto, se establece a través del puerto 21.

La conexión de datos

Conecta el protocolo de transferencia de datos del cliente, que es quien accede al sistema de archivos del cliente, con el protocolo de transferencia de datos del servidor, que es quien accede al sistema de archivos del servidor. Por esta conexión viajan los archivos y puede darse el caso de que viajen datos en ambas direcciones de forma simultánea. Por defecto, se establece a través del puerto 20.

2.2. Modos de conexión

A. Modo activo



Esquema 4.

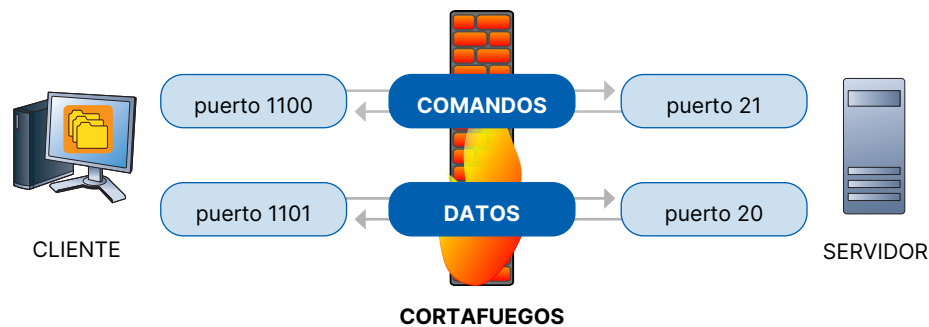
Como puede verse en el esquema anterior:

- » El cliente inicia la conexión de control al puerto 21 del servidor utilizando un puerto local mayor que 1023.
- » El servidor establece la conexión de datos. Para ello, utiliza su puerto 20 para conectar al siguiente puerto del que ha utilizado el cliente.

En el caso de querer utilizar este modo, se deberá configurar el router o el cortafuegos para que permitan este tipo de conexiones hacia el cliente.

B. Modo pasivo

En este caso, es el cliente quien inicia tanto la conexión de control como la de datos.



Esquema 5.

Como puede verse en el esquema anterior:

- » El cliente establece la conexión de control al puerto 21 del servidor utilizando un puerto local mayor que 1023.
- » El cliente espera que el servidor le indique en qué puerto debe realizar la conexión de datos.
- » El cliente inicia la conexión de datos al puerto que el servidor le ha indicado utilizando el puerto local siguiente al que ha utilizado para la conexión de control.

En este caso, ya no es necesario configurar el router o el cortafuegos del cliente para que admitan conexiones entrantes, ya que el cliente no recibirá ninguna conexión de este tipo.

2.3. Transferencia de información

Cuando se intercambian archivos entre diferentes máquinas, puede ocurrir que estas tengan distintas representaciones de los datos, por lo que puede que sea necesario efectuar ciertas transformaciones después de leer los de la máquina de origen y antes de guardarlos en la máquina de destino. Por este motivo, al comenzar la comunicación, se puede indicar qué tipo de datos se van a transferir y puede elegirse entre ASCII o binario.

- » **Tipo ASCII:** es el tipo por defecto y se utiliza cuando se quieren transferir archivos de texto. La máquina que contiene la información que queremos transferir la transforma de su representación interna a la representación estándar NVT-ASCII de 8 bits y la envía. La máquina de destino debe llevar a cabo la operación inversa, es decir, al recibir la información la convierte de la representación estándar NVT-

ASCII de 8 bits a la representación interna que utilice y, una vez convertida, la almacena.

» **Tipo binario:** es el tipo que se debe utilizar para transferir cualquier archivo que no sea texto.

2.4. Códigos de respuesta

Los códigos de respuesta sirven para verificar que las acciones que están ejecutándose corresponden a las peticiones recibidas y para garantizar que el proceso del cliente siempre conoce el estado del servidor. Por tanto, se debe generar al menos un código de respuesta por cada petición. Los códigos de respuesta son numéricos de tres cifras y, dependiendo del valor de la primera cifra, se pueden agrupar en cinco clases:

LA PRIMERA CIFRA ES 1: RESPUESTA PRELIMINAR POSITIVA

Se ha aceptado la petición y se espera una respuesta para poder seguir. Por ejemplo, podría indicar que el servidor ha aceptado la petición y el cliente debe transmitir por el canal de datos. Algunos códigos de este grupo pueden ser los siguientes:

- » **110:** indica que se puede reanudar la respuesta.
- » **120:** indica que el cliente debe esperar hasta que el servidor esté preparado.

LA PRIMERA CIFRA ES 2: RESPUESTA DE FINALIZACIÓN POSITIVA

La petición se ha completado satisfactoriamente y el servidor está listo para recibir otra petición. Algunos códigos de este grupo pueden ser los siguientes:

- » **200:** se ha completado la operación.
- » **220:** el servidor está preparado.
- » **230:** inicio de sesión correcto.

LA PRIMERA CIFRA ES 3: RESPUESTA INTERMEDIA POSITIVA

Petición aceptada, pero, para poder llevarla a cabo, el cliente debe enviar más información. Algunos códigos de este grupo pueden ser los siguientes:

- » **331:** se solicita la contraseña para poder iniciar la sesión.
- » **350:** indica que se debe enviar el comando siguiente para poder completar la operación.

LA PRIMERA CIFRA ES 4: RESPUESTA DE FINALIZACIÓN NEGATIVA TRANSITORIA

La petición ha sido rechazada temporalmente y no se ha llevado a cabo ninguna acción. El cliente puede volver a realizar la petición. Algunos códigos de este grupo pueden ser los siguientes:

- » **421:** indica que el servicio no está disponible.
- » **425:** indica que no ha sido posible abrir la conexión de datos.

LA PRIMERA CIFRA ES 5: RESPUESTA DE FINALIZACIÓN NEGATIVA PERMANENTE

La petición se ha rechazado y no se ha llevado a cabo ninguna acción. El cliente no debería volver a efectuar la petición. Algunos códigos de este grupo pueden ser los siguientes:

- » **530:** la autenticación del usuario no ha sido satisfactoria.
- » **550:** indica que no se puede acceder al archivo que el cliente está solicitando, bien porque no existe, bien porque el usuario no tiene suficientes permisos.

Esquema 6.

2.5. Tipo de acceso

El acceso al servidor FTP desde el cliente puede ser de dos tipos, anónimo o autorizado.

» **Acceso anónimo:** el cliente no necesita ningún tipo de autenticación para establecer la conexión con el servidor y acceder al sistema de archivos. Las operaciones que podrá hacer sobre los archivos y directorios del servidor dependerán de los permisos asignados a estos por el propio sistema de archivos del servidor. Hay que tener mucho cuidado si se habilita este tipo de acceso al servidor y establecer muy bien los permisos del directorio al que se conectará, como también los de sus subdirectorios y archivos. Por ejemplo, si se permitiera la escritura, el usuario podría subir al servidor archivos maliciosos. Por ello, normalmente, solo se permite la lectura de archivos para que los usuarios descarguen programas o utilidades.

» **Acceso autorizado:** el cliente necesita disponer de un usuario y una contraseña para poder establecer la conexión con el servidor. En este caso, dependiendo de la configuración que el administrador del servidor haya definido, cada usuario podrá tener acceso a distintos directorios y con diferentes permisos sobre los archivos contenidos en ellos.

2.6. Clientes

Como se ha visto antes, el usuario puede conectarse de diferentes formas al servidor. La primera de ellas es a través de una ventana de comandos. La segunda forma es a través de un programa con entorno gráfico. Hasta hace unos años, había una tercera forma de conexión, y era a través de los navegadores de Internet, pero, hoy en día, en las versiones más recientes de los navegadores actuales, ya no es posible.

CLIENTES DEDICADOS EN MODO COMANDO

Se utiliza a través del terminal de la máquina. En este caso, el usuario deberá conocer los comandos asociados a las órdenes que desee ejecutar. Una vez establecida la conexión, se podrá indicar si se quiere transferir información de tipo ASCII o binario. Si no se indica nada, el tipo por defecto es ASCII.

Estos son algunos de los comandos interactivos:

- » **cd:** cambio de directorio remoto.
- » **lcd:** cambio de directorio local.
- » **get:** transfiere un archivo remoto al sistema local.
- » **put:** transfiere un archivo local al sistema remoto.
- » **quit, exit y bye:** salen de **ftp**.

El listado completo de comandos depende del cliente instalado y se puede obtener a partir de los comandos *help* o *?*. La siguiente captura de pantalla está realizada desde una ventana de comandos de Windows 10:

1

```

C:\Users\begop>ftp
ftp> help
Los comandos se pueden abreviar. Comandos:

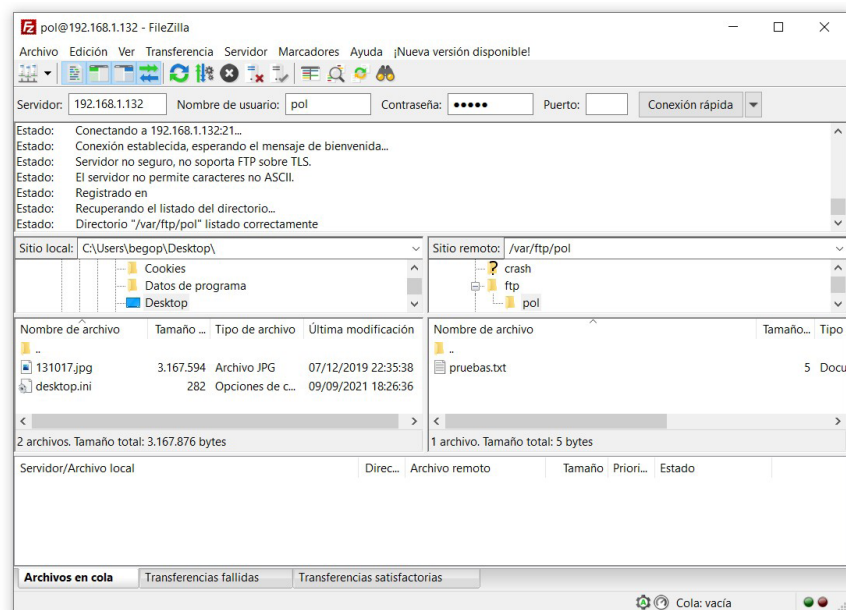
!      delete      literal      prompt      send
?      debug       ls           put          status
append dir           mdelete     pwd          trace
ascii  disconnect   mdir        quote        type
bell   get           mget        quote        user
binary glob          mkdir       recv         verbose
bye    hash          mls         remotehelp
cd     help          mput        rename
close lcd           open        rmdir
ftp>

```

CLIENTES DEDICADOS EN MODO GRÁFICO

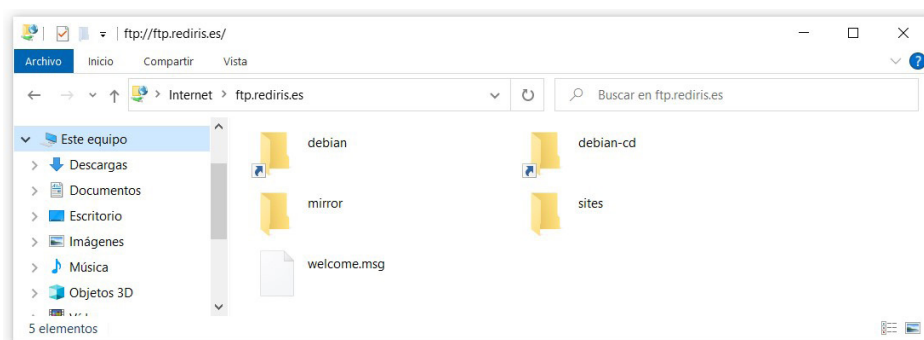
Este modo lo facilita un programa que seguramente tendrá que instalarse en la máquina. En este caso, el usuario no necesita conocer los comandos, pero sí saber manejar el programa. La mayoría de los programas actuales ya detectan de forma automática si el tipo de los archivos por transferir es de tipo ASCII o binario. Un ejemplo de un cliente con entorno gráfico es FileZilla. A continuación, puede verse una captura de pantalla de su aspecto:

2



OTROS CLIENTES

Hay otros programas pensados para realizar otras funciones (como, por ejemplo, el Explorador de archivos) que pueden utilizarse también para conectar a un servidor FTP y permitir el intercambio de archivos. Esta opción es una solución intermedia entre las dos anteriores, ya que resulta mucho más cómoda de utilizar que el modo comandos, pero no tiene tantas prestaciones como los clientes dedicados en modo gráfico que, entre algunas de sus funciones, ofrecen la posibilidad de poder guardar varias conexiones. La conexión se realiza a través de la ruta de la ventana del Explorador de archivos con el formato siguiente: `ftp://IP_servidor` o `ftp://nombre_servidor`. En el caso de que el servidor admita la conexión anónima, una vez establecida la conexión con el servidor, la ventana mostrará los directorios y archivos correspondientes. Si la conexión requiere autenticación, se mostrará una ventana donde se solicita el usuario y la contraseña. La siguiente captura muestra la conexión realizada a `ftp://ftp.rediris.es/` en modo anónimo utilizando el Explorador de archivos de Windows 10:



Esquema 7.

2.7. Servidores

En el servidor es donde se configuran algunos de los aspectos descritos en los puntos anteriores. Existen muchos servidores, tanto de código libre como de código propietario. En todos ellos se pueden indicar los parámetros de configuración, como, por ejemplo, si se permite o no el modo anónimo, el directorio por defecto al que llegará el usuario que se conecte o el modo de conexión que utilizar. Algunos servidores de código libre son ProFTPD y vsftpd.

2.7. Seguridad

El protocolo FTP no es seguro, ya que tanto la información de autenticación —es decir, el usuario y la contraseña— como los propios archivos que se están transmitiendo viajan sin cifrar. Cuando se diseñó el protocolo, se dio prioridad a que la transferencia de archivos fuera rápida antes que segura.

Para solucionar este problema, se creó FTPS. Realmente, no se trata de un protocolo, sino del uso de FTP sobre el protocolo **TLS** (*Transport Layer Security*) o, en sus orígenes, sobre **SSL** (*Secure Sockets Layer*), el antecesor de TLS. Su funcionamiento consiste en introducir una nueva capa que implementa el cifrado entre cliente y servidor. De esta manera, se incrementa la **seguridad** en las comunicaciones. No hay que confundir FTPS con SFTP. SFTP es un protocolo que se estudiará en el siguiente apartado.

2 CASO PRÁCTICO



CONFIGURACIÓN AVANZADA DE SFTP

Introducción

La configuración por defecto del servidor OpenSSH permite que cualquier usuario con permisos para conectarse sea capaz de transferir archivos entre el cliente y el servidor. Normalmente, esto no supone un problema.

Lo que no siempre es aconsejable es que un usuario, por el hecho de tener permisos para hacer transferencias de archivos en un servidor mediante SFTP, pueda obtener un *shell* del sistema gracias a SSH. Otro problema derivado de la configuración por defecto de OpenSSH es que cualquier usuario podría descargar gran cantidad de archivos con contenido delicado, como, por ejemplo, la mayoría de los archivos de configuración del sistema.

En esta tarea vas a aprender a configurar SFTP de manera que los usuarios que pertenecen al grupo *almacenamiento* tengan acceso a SFTP, pero sin poder obtener un *shell* en el servidor y, además, no puedan acceder a la información almacenada fuera de su directorio personal.

Claves de resolución

- » Preparación del grupo y un par de usuarios de pruebas del sistema.
- » Configuración del servidor OpenSSH con *internal-sftp* para que los usuarios del grupo creado no puedan abrir un *shell* del sistema y solo puedan acceder a su directorio personal para transferir archivos.
- » Comprobación del funcionamiento.

SOLUCIÓN

Preparación del grupo y los usuarios

1. Accede al **servidor Ubuntu** y abre una sesión con un usuario administrador.
2. Para poder enjaular a los usuarios que se conecten por SFTP (en adelante, *usuarios SFTP*) de manera que no puedan acceder fuera de su directorio personal, este debe pertenecer al usuario *root*, por lo que vas a crear un directorio que pertenece a dicho usuario y será el directorio personal de todos los usuarios SFTP. Solo *root* tendrá permisos de escritura sobre este directorio.

```
admin01@server:~$ cd /home
admin01@server:/home$ sudo mkdir almacenamiento
admin01@server:/home$ ll almacenamiento
total 8
drwxr-xr-x 2 root root 4096 jul 19 09:01 ./
drwxr-xr-x 6 root root 4096 jul 19 09:01 ../
admin01@server:/home$
```

3. Vas a crear un par de usuarios SFTP del grupo *almacenamiento*, pero no podrán obtener un *shell* local ni a través de SSH.

```
admin01@server:/home$ sudo addgroup almacenamiento
Adding group 'almacenamiento' (GID 1003) ...
Done.
admin01@server:/home$ sudo adduser --shell /usr/sbin/nologin --home /
home/almacenamiento --ingroup almacenamiento --gecos "" almacen01
Warning: The home dir /home/almacenamiento you specified already exists.
Adding user 'almacen01' ...
Adding new user 'almacen01' (1003) with group 'almacenamiento' ...
The home directory '/home/almacenamiento' already exists. Not copying
from '/etc/skel'.
adduser: Warning: The home directory '/home/almacenamiento' does not
belong to the user you are currently creating.
New password:
Retype new password:
passwd: password updated successfully
admin01@server:/home$ sudo adduser --shell /usr/sbin/nologin --home
/home/almacenamiento --ingroup almacenamiento --gecos "" almacen02
Warning: The home dir /home/almacenamiento you specified already
exists.
[...]
```

```
admin01@server:/home$ tail -2 /etc/passwd
almacen01:x:1003:1003:,,,:/home/almacenamiento:/usr/sbin/nologin
almacen02:x:1004:1003:,,,:/home/almacenamiento:/usr/sbin/nologin
admin01@server:/home$
```

4. Como los usuarios SFTP no podrán escribir sobre su propio directorio personal, debes crear, dentro de este, un directorio para cada usuario. Dependiendo del uso que se le dé y del nivel de confianza, los permisos de estos directorios se configurarán de una u otra manera. En esta ocasión, vas a configurarlos para que solo el propietario pueda acceder a ellos.

```
admin01@server:/home$ sudo mkdir almacenamiento/almacen01
admin01@server:/home$ sudo mkdir almacenamiento/almacen02
admin01@server:/home$ sudo chmod 700 almacenamiento/almacen01
admin01@server:/home$ sudo chmod 700 almacenamiento/almacen02
admin01@server:/home$ sudo chown almacen01:almacenamiento
almacenamiento/almacen01
admin01@server:/home$ sudo chown almacen02:almacenamiento
almacenamiento/almacen02
```

5. Instala el programa *tree* y comprueba si la estructura de directorios es correcta.

```
admin01@server:/home$ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Se instalarán los siguientes paquetes NUEVOS:
  tree
[...]

admin01@server:/home$ sudo tree -ugp almacenamiento/
[drwxr-xr-x root    root  ] almacenamiento/
├─ [drwx----- almacen01 almacenamiento] almacen01
└─ [drwx----- almacen02 almacenamiento] almacen02

2 directories, 0 files
admin01@server:/home$
```

Configuración del servidor OpenSSH

6. Haz una copia de seguridad del archivo de configuración del servidor OpenSSH, editálo y añade en la parte final la configuración necesaria para que:

- » se sustituya el subsistema *sftp* actual por *internal-sftp*. Asegúrate de que solo haya activa una palabra clave *Subsystem* en el archivo de configuración;
- » se aplique la siguiente configuración a los usuarios que pertenezcan al grupo *almacenamiento*:
 - el usuario solo podrá acceder a su carpeta personal y a sus subcarpetas;
 - no permitirá el reenvío X11 ni el TCP;
 - no se permitirá la asignación de un emulador de terminal (PTY);
 - se forzará la ejecución del comando *internal-sftp*.

```
admin01@server:/etc/ssh$ cd /etc/ssh
admin01@server:/etc/ssh$ sudo cp sshd_config sshd_config.20221719
admin01@server:/etc/ssh$ sudo vi sshd_config
[...]
```

```
#Subsystem      sftp    /usr/lib/openssh/sftp-server
Subsystem      sftp    internal-sftp
```

```
Match Group almacenamiento
    ChrootDirectory %h
    X11Forwarding no
    AllowTcpForwarding no
    PermitTTY no
    ForceCommand internal-sftp
```

7. Reinicia el servidor OpenSSH y comprueba su estado.

```
admin01@server:/etc/ssh$ sudo systemctl restart sshd.service
admin01@server:/etc/ssh$ sudo systemctl status sshd.service

• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2022-07-19 14:54:44 UTC; 4s ago
[...]
```

```
admin01@server:/etc/ssh$
```

Configuración del funcionamiento

8. Ahora vas a trabajar desde un **equipo cliente**; puedes usar tu propio ordenador o una máquina virtual con, por ejemplo, Ubuntu Desktop.
9. Comprueba con uno de los usuarios SFTP creados que no puede conectarse vía *ssh* ni realizar transferencias con *scp*.

```

~$ ssh almacen01@192.168.0.192
almacen01@192.168.0.192's password:
PTY allocation request failed on channel 0
This service allows sftp connections only.
Connection to 192.168.0.192 closed.
~$ scp saludo.txt almacen01@192.168.0.192:almacen01
almacen01@192.168.0.192's password:
This service allows sftp connections only.
~$

```

10. Comprueba ahora que este usuario sí puede establecer una conexión interactiva con *sftp* y transferir archivos, pero siempre dentro de su directorio personal y sus correspondientes subdirectorios.

```

~$ sftp almacen01@192.168.0.192
almacen01@192.168.0.192's password:
Connected to 192.168.0.192.
sftp> pwd
Remote working directory: /
sftp> cd ..
sftp> pwd
Remote working directory: /
sftp>
sftp> ls
almacen01 almacen02 sftp> cd almacen01
sftp> put saludo.txt
Uploading saludo.txt to /almacen01/saludo.txt
saludo.txt                               100% 5      5 . 7 K B / s
00:00 sftp> ls
salir.php saludo.txt sftp> quit
~$

```

11. Por último, asegúrate de que los usuarios que no pertenecen al grupo *almacenamiento* pueden realizar conexiones con *ssh* y transmitir archivos tanto con *scp* como con SFTP y sin limitarse a su directorio personal.

```

~$ ssh usuario01@192.168.0.192
usuario01@192.168.0.192's password:
Welcome to Ubuntu 22.04 LTS (GNU/Linux 5.15.0-41-generic x86_64)
[...]

Last login: Sat Jul 16 15:15:26 2022 from 192.168.0.70
usuario01@server:~$ exit

```

```

logout
Connection to 192.168.0.192 closed.
~$ scp saludo.txt usuario01@192.168.0.192:
usuario01@192.168.0.192's password:
saludo.txt                                100% 5      4 . 5 K B / s
00:00 ~$ ~$ sftp usuario01@192.168.0.192
usuario01@192.168.0.192's password:
Connected to 192.168.0.192.
sftp> cd /
sftp> ls
bin          boot        dev         etc         home       hil
lib32        lib64       libx32      lost+found  media      tm
opt          proc        root        run         sbin       p
srv          sys         tmp         usr         var        sftp>
quit
~$

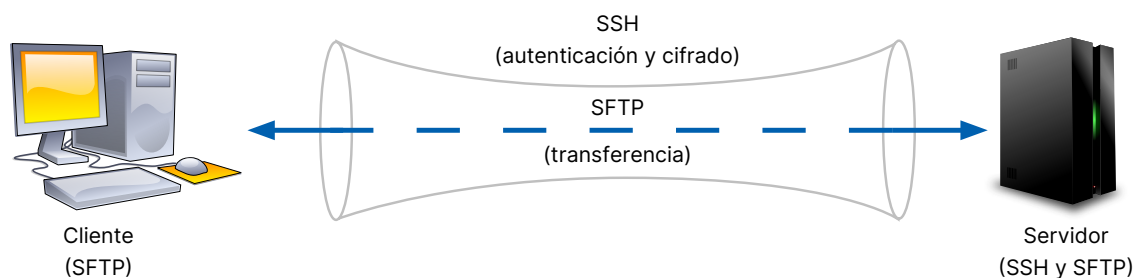
```

03 Protocolo SFTP

Hoy en día no es necesario acudir a la prensa especializada para encontrar noticias sobre ataques informáticos a empresas, organizaciones o, incluso, estados. Esto ha hecho que crezca la preocupación por la seguridad, por lo que se ha incrementado el número de medidas, pasivas y activas, implementadas tanto en los equipos informáticos como en las redes de ordenadores. En el caso de las redes, la primera medida de seguridad adoptada suele ser asegurar la transferencia de archivos, de la que se va a tratar en este apartado. Como ya se ha visto, el protocolo SFTP (*SSH File Transfer Protocol*) ofrece dos servicios principales: la gestión de un **sistema de archivos remoto** y la **transferencia segura de archivos**. Es importante reseñar que este protocolo deja en manos externas tres aspectos muy importantes de la seguridad:

- » La gestión de la **autenticación** de los intervinientes en la comunicación. Esta autenticación puede ser del cliente ante el servidor y viceversa.
- » La **protección** de la información en tránsito para intentar garantizar tanto su privacidad, como su integridad.
- » El **control de acceso** al sistema de archivos.

Es por esta razón por la que se asume que el protocolo SFTP trabaja sobre un canal seguro, normalmente, con SSH versión 2. Pero hay que tener en cuenta que este aspecto no implica que no sea un protocolo independiente, de hecho, puede trabajar con otros protocolos, por ejemplo, para implementar transferencia segura de archivos a través de TLS o para gestionar VPN.



Esquema 8.

Cabe destacar que el IETF (*Internet Engineering Task Force*) no tiene ninguna RFC sobre el protocolo SFTP, sino que ofrece una serie de borradores (*Internet-Draft*) que tienen una vigencia limitada de seis meses y suelen ser actualizados, reemplazados o declarados obsoletos con mayor frecuencia que otros tipos de documentos, como los RFC. Aunque actualmente hay trece borradores publicados y la última versión de SFTP es la 6, la mayoría de los servidores trabajan con la versión 3, descrita en el borrador 2.

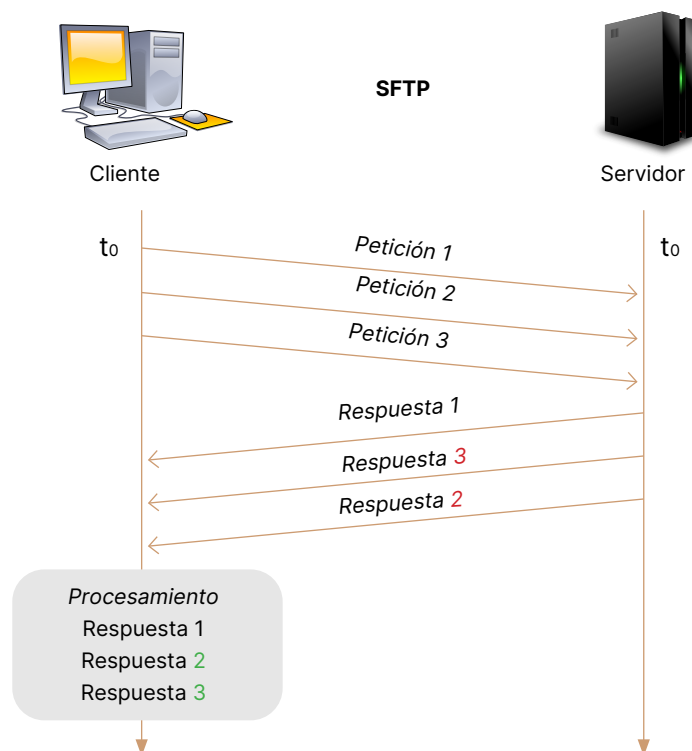
En la siguiente tabla hay una comparativa entre los protocolos FTP y SFTP.

	FTP	SFTP
CREDENCIALES	Transmitidas sin cifrar.	Gestionadas y cifradas por el canal seguro.
ARCHIVOS	Transmitidos sin cifrar.	Cifrados por el canal seguro.
CANALES	Dos canales: control y datos.	Un solo canal para control y datos.
PUERTO	20/TCP (datos) y 21/TCP (control).	22/TCP (SSH).

Tabla 2.

3.1. Funcionamiento de SFTP

El funcionamiento básico de este protocolo responde al modelo **cliente-servidor**, basado en peticiones realizadas por los clientes que son atendidas por el servidor, así como en las respuestas que este les ofrece a los clientes. El servidor puede atender muchas peticiones, por lo que las respuestas deben incluir el mismo identificador que la petición a la que corresponden, así, el cliente será capaz de relacionarlas incluso cuando se le envíen desordenadas y las procesará en el orden correcto.



Esquema 9.

Las peticiones que puede realizar un cliente que usa SFTP versión 3 son:

- » Abrir, crear y cerrar archivos. Los archivos pueden abrirse en modo lectura o lecturaescritura.
- » Leer y escribir archivos.
- » Eliminar y cambiar el nombre de archivos.
- » Obtener y configurar los atributos de archivos.
- » Listar, crear y eliminar directorios.
- » Gestionar enlaces y rutas.

Por su parte, las respuestas del servidor consisten en facilitar la siguiente información:

- » el identificador de un archivo;
- » los datos de un archivo;
- » los nombres de archivos de un directorio;
- » los atributos de un archivo.

Las respuestas del servidor incluyen el estado, que ofrece información sobre el resultado de la petición que ha realizado el cliente. Este estado indica que todo ha ido bien o, en caso de encontrarse con un error, de cuál se trata. Algunos de estos errores son los siguientes:

- » no se ha encontrado el archivo solicitado;
- » se ha encontrado el final de archivo, por lo que no hay más datos;
- » no se tienen permisos para efectuar cierta operación sobre el archivo indicado;
- » se ha perdido o no hay conexión con el servidor.

3.2. SFTP con OpenSSH

Son muchas las ocasiones en las que un administrador de un sistema quiere controlarlo remotamente. Para poder desempeñar esta labor, existen muchos métodos, pero uno de los más extendidos y seguros es el protocolo **SSH** (*Secure Shell*). Este protocolo proporciona un canal seguro entre los participantes en la comunicación porque, además de cifrar la información gracias a que utiliza técnicas criptográficas, valida tanto al cliente ante el servidor como al servidor ante el cliente, evitando así que uno u otro puedan ser suplantados. El puerto TCP asignado al servicio SSH es el 22.

Una de las implementaciones del protocolo SSH más usadas es **OpenSSH**, que, además del servidor y el cliente SSH que permiten controlar equipos remotos, también ofrece la transferencia de archivos a través de SCP y SFTP. De esta manera, aunque principalmente se usa para controlar un servidor remoto, también se utiliza muy a menudo para transferir datos entre cliente y servidor de forma segura, gestionar claves RSA que permitan el acceso al servidor sin introducir la contraseña y redirigir los datos de cualquier aplicación utilizando un túnel seguro. Estas son algunas de las características más importantes de OpenSSH:

- » es un proyecto de código abierto;
- » el reenvío de puertos permite establecer canales seguros entre los dispositivos para que puedan usarlos protocolos no seguros;
- » admite validación por contraseña y por claves públicas y privadas;
- » permite la compresión de datos.

SCP

Con la aplicación de consola **scp** suministrada por OpenSSH se pueden transferir archivos entre un cliente y un servidor en ambos sentidos, pero, además, permite que un cliente controle la transferencia directa de archivos entre dos servidores remotos sin que la información pase por el equipo cliente. La aplicación scp requiere de la ejecución de un *shell* a través de SSH y usa su misma autenticación.

Su sintaxis básica es **scp [-i *archivo_de_identidad*] *fuente destino***, donde **fuente** y **destino** suelen representar un archivo local o uno remoto con la forma **[usuario@]host: [ruta]**, mientras que el archivo de identidad indica la clave privada que se usará para la validación del usuario. Los elementos entre corchetes son opcionales.

En el siguiente ejemplo se puede ver cómo se copia un archivo remoto al sistema de archivos local con un nombre de archivo distinto al original y, además, se indica que se guarde en un directorio distinto al actual.

```
~$ scp usuario01@192.168.0.192:imagenes/2Vacas.jpg Descargas/
vacas.jpg
usuario01@192.168.0.192's password:
2Vacas.jpg                100% 2040KB 104.8MB/s   00:00
~$
```

SERVIDOR SFTP

Respecto al protocolo **SFTP**, OpenSSH ofrece tanto el servidor como el cliente para la transmisión de datos. Estos usarán la conexión SSH tanto para la validación como para la protección de los archivos.

El **servidor SFTP** no actúa de manera independiente, sino que se usa junto al servidor SSH. Es decir, el servidor OpenSSH invoca al subsistema configurado cuando quiere atender las peticiones de transferencia de archivos de un cliente SFTP. En *sshd_config*, el archivo de configuración del servidor de OpenSSH, se puede escoger entre el subsistema *sftp-server* e *internal-sftp* y configurarlos de acuerdo a las necesidades del sistema. Estas son algunas de las palabras clave que se pueden usar en este archivo de configuración para gestionar la transferencia de archivos y la conexión de control remoto:

» **Subsystem:** configura un subsistema externo, por ejemplo, para la transferencia de archivos. Se debe especificar el subsistema y el comando que lo lanza. Por ejemplo, *Subsystem sftp sftp-server*.

» **Match:** si se cumplen todas las condiciones especificadas en la misma línea, se aplicarán las palabras clave hasta el próximo bloque *Match* o hasta el fin del archivo. Hay que tener en cuenta que si una palabra clave se especifica en un bloque *Match* y ya se ha usado con anterioridad, prevalece la configuración escrita dentro del bloque. Además, si una palabra clave aparece en más de un bloque *Match*, prevalece el valor especificado en el primero de ellos. Se puede obtener una lista de todas las palabras clave que admite un bloque *Match* a través del comando **man**, el manual de referencia en sistemas GNU/Linux.

» **ChrootDirectory:** permite enjaular la conexión de manera que solo se pueda acceder a los archivos y directorios que se encuentren dentro de la ruta especificada. El directorio indicado debe pertenecer al usuario *root* y ningún otro usuario ni grupo podrá tener permisos de lectura, aunque dentro de este podrán existir archivos y directorios con otros permisos, incluso con otros propietarios. Su valor por defecto es **none**, lo que indica que no se limitará el acceso al sistema de archivos remoto. Esta palabra clave acepta los siguientes *tokens* en el valor asignado:

- **%:** el carácter %;
- **%h:** el directorio personal del usuario;
- **%U:** el identificador de usuario numérico. Puede obtenerse del archivo de configuración */etc/passwd*;
- **%u:** el nombre de usuario.

» **X11Forwarding:** indica si se permite o no el reenvío X11, una interfaz gráfica muy extendida en los sistemas GNU/Linux. Es independiente del núcleo del sistema operativo, está basado en el modelo cliente-servidor y puede trabajar a través de la red. El valor por defecto es **no**.

» **AllowTcpForwarding:** se impide que los clientes realicen un reenvío de tráfico TCP. De esta manera, si también se niega a los clientes el acceso a un *shell*, aumenta la seguridad del sistema. El valor por defecto es **no**, pero también admite los valores **yes**, **all**, **local** y **remote**.

» **PermitTTY:** indica si se permitirá o no la asignación de un emulador de terminal (PTY) a los clientes. El valor por defecto es **yes**.

» **ForceCommand:** no se tiene en cuenta el comando suministrado por el usuario y se fuerza la ejecución del comando especificado. En el caso de indicar el comando **internal-sftp**, se facilitará la configuración en caso de querer enjaular las conexiones SFTP.

A continuación, se muestra parte del contenido que se incluye en el archivo de configuración por defecto del servidor OpenSSH. Esta configuración establece el subsistema *sftp-server* para la transferencia segura de archivos y modifica la configuración para el usuario *anoncvs*.

[...]

```
Subsystem      sftp      /usr/lib/openssh/sftp-server
Match User anoncvs
    X11Forwarding no
    AllowTcpForwarding no
    PermitTTY no
    ForceCommand cvs server
```

CLIENTE SFTP

OpenSSH incluye un cliente SFTP en la instalación del cliente SSH, así que, desde cualquier equipo donde se haya instalado este último, además de controlar un servidor remoto, se podrán transferir archivos. Este cliente, llamado *sftp*, ofrece una interfaz interactiva en modo texto similar a la que ofrecen los clientes FTP en modo texto.

La sintaxis básica para establecer la conexión es **sftp [-i *archivo_de_identidad*] *destino***, donde *destino* se especifica con la forma [*usuario@*]*host*[:*ruta*], mientras que el archivo de identidad indica la clave privada que se usará para la validación del usuario.

Además del cliente *sftp*, el servidor SFTP de OpenSSH permite la conexión de otros programas como otros clientes SFTP, clientes FTP compatibles con SFTP o administradores de archivos que soporten este protocolo.

En este ejemplo de conexión SFTP se muestra la lista de comandos disponibles, sus argumentos y una explicación breve de cada uno de ellos. Cabe destacar la similitud con los comandos disponibles en FTP.

```
~$ sftp admin01@192.168.0.192
admin01@192.168.0.192's password:
Connected to 192.168.0.192.
sftp> ?
Available commands:
bye                               Quit sftp
cd path                           Change remote directory to 'path'
chgrp [-h] grp path              Change group of file 'path' to 'grp'
chmod [-h] mode path             Change permissions of file 'path' to
'mode'
chown [-h] own path              Change owner of file 'path' to 'own'
df [-hi] [path]                  Display statistics for current
directory or
filesystem containing 'path'
exit                              Quit sftp
get [-afpR] remote [local]       Download file
help                             Display this help text
lcd path                         Change local directory to 'path'
lls [ls-options] [path]]         Display local directory listing
lmkdir path                      Create local directory
ln [-s] oldpath newpath          Link remote file (-s for symlink)
lpwd                             Print local working directory
ls [-lafhlNrSt] [path]           Display remote directory listing
lumask umask                     Set local umask to 'umask'
mkdir path                       Create remote directory
progress                         Toggle display of progress meter
put [-afpR] local [remote]       Upload file
pwd                              Display remote working directory
quit                             Quit sftp
```

3

<code>reget [-fpR] remote [local]</code>	Resume download file
<code>rename oldpath newpath</code>	Rename remote file
<code>reput [-fpR] local [remote]</code>	Resume upload file
<code>rm path</code>	Delete remote file
<code>rmdir path</code>	Remove remote directory
<code>symlink oldpath newpath</code>	Symlink remote file
<code>version</code>	Show SFTP version
<code>!command</code>	Execute 'command' in local shell
<code>!</code>	Escape to local shell
<code>?</code>	Synonym for help
<code>sftp> exit</code>	
<code>~\$</code>	

Esquema 10.

3.3. Sistema de archivos SSHFS

Este sistema de archivos cliente está basado en SFTP y FUSE (*Filesystem in Userspace*). Por un lado, gracias a SFTP, permite montar parte del sistema de archivos remoto de un servidor SSH sobre un directorio local, así que los archivos remotos se podrán gestionar como si fueran locales. Esto hace que aplicaciones que no están preparadas para trabajar con archivos remotos puedan hacerlo porque los ven como si fueran locales. Por otro lado, usar FUSE permite que los usuarios sin permisos de administración puedan gestionar los puntos de montaje.

La sintaxis de esta aplicación es `sshfs [usuario@]host:[directorio_remoto] punto_de_montaje_local [opciones]`, mientras que, para desmontar el recurso remoto, se debe ejecutar `fusermount -u punto_de_montaje_local`.



VÍDEOLAB

Para consultar el siguiente vídeo sobre autenticación de aplicaciones, escanea el código QR o [pulsa aquí](#).

UAX FP